

Nama : Hanif Adira Putra
Kelompok : Bismuth

Pseudocode Problem Set 1 : The Last Astronaut

MULAI

DEKLARASI N, K, cur, countAlive, idx, step, eliminatedNumber, orderCount, survivor : integer
DEKLARASI alive[0..N-1] : boolean DEKLARASI order[0..N-1] : integer

BACA N

BACA K

UNTUK $i = 0$ SAMPAI $N-1$ LAKUKAN $alive[i] = true$
AKHIR UNTUK

$countAlive = N$ $cur = 0$
 $orderCount = 0$

SELAMA $countAlive > 1$ LAKUKAN

$idx = cur$

$step = 1$

 SELAMA $step < K$ LAKUKAN

$idx = (idx + 1) \text{ MOD } N$

 SELAMA $alive[idx] = false$ LAKUKAN

$idx = (idx + 1) \text{ MOD } N$ AKHIR SELAMA

$step = step + 1$

 AKHIR SELAMA

$eliminatedNumber = idx + 1$ $order[orderCount] = eliminatedNumber$ $orderCount =$
 $orderCount + 1$ $alive[idx] = false$
 $countAlive = countAlive - 1$

JIKA $countAlive > 0$ MAKA $cur = (idx + 1) \text{ MOD } N$

 SELAMA $alive[cur] = false$ LAKUKAN

$cur = (cur + 1) \text{ MOD } N$

 AKHIR SELAMA AKHIR JIKA

JIKA $eliminatedNumber \text{ MOD } 2 = 0$ MAKA

$K = K + 2$

SELAIN

$K = K - 1$

AKHIR JIKA

JIKA $K < 2$ MAKA

$K = 2$

AKHIR JIKA
AKHIR SELAMA

UNTUK $i = 0$ SAMPAI $N-1$ LAKUKAN

JIKA $alive[i] = \text{true}$ MAKA $survivor = i + 1$

AKHIR JIKA
AKHIR UNTUK

TULIS *order*
TULIS *survivor*

SELESAI

Penjelasan Logika

Logika penyelesaian Problem 1 didasarkan pada simulasi lingkaran astronot yang direpresentasikan sebagai array boolean *alive* berukuran N , di mana indeks ke- i mewakili astronot bernomor $i+1$. Variabel *cur* menandai titik awal hitungan pada setiap putaran eliminasi. Untuk menemukan astronot yang tereliminasi, program bergerak sebanyak $(K-1)$ langkah dari *cur* sambil melewati posisi yang sudah tidak hidup, karena posisi awal itu sendiri sudah dihitung sebagai hitungan pertama. Pencarian posisi hidup berikutnya dilakukan melalui perulangan dengan operasi modulo N agar pergerakan tetap melingkar ketika mencapai akhir array.

Setelah astronot pada posisi *idx* ditemukan, nomor astronotnya dicatat ke dalam array *order* sebagai bagian dari urutan eliminasi, statusnya diubah menjadi tidak hidup, dan jumlah astronot yang tersisa (*countAlive*) dikurangi satu. Titik awal untuk putaran berikutnya (*cur*) kemudian ditentukan sebagai astronot hidup pertama setelah posisi yang baru saja dieliminasi. Nilai K selanjutnya diperbarui sesuai aturan: bertambah 2 jika nomor yang dieliminasi genap, atau berkurang 1 jika ganjil, dengan batas bawah K tidak boleh kurang dari 2. Seluruh proses ini diulang di dalam perulangan utama hingga hanya tersisa satu astronot, yang kemudian dicari dan dilaporkan sebagai penyintas terakhir bersama seluruh urutan eliminasinya.

Pseudocode Problem Set 2 : Alien In The Middle

MULAI

DEKLARASI `message[0..MAX-1]` : character

DEKLARASI `origVal[0..MAX-1]`, `result[0..MAX-1]` : integer/character

DEKLARASI `len`, `i`, `newVal` : integer

BACA `message`

`len = 0`

SELAMA `message[len] != KARAKTER_KOSONG` LAKUKAN `len = len + 1`

AKHIR SELAMA

UNTUK `i = 0` SAMPAI `len-1` LAKUKAN

`origVal[i] = NILAI_POSISI_HURUF(message[i])` { A=1, B=2, ..., Z=26 }

AKHIR UNTUK

UNTUK `i = 0` SAMPAI `len-1` LAKUKAN

JIKA `i = 0` MAKA

`newVal = origVal[i]` SELAIN

`newVal = origVal[i] + origVal[i-1]` JIKA `newVal > 26` MAKA `newVal = newVal - 26`

AKHIR JIKA AKHIR JIKA

`result[i] = HURUF_KE(newVal)` { `newVal=1` -> 'A', ..., `newVal=26` -> 'Z' }

AKHIR UNTUK

TULIS `result`

SELESAI

Penjelasan Logika

Penyelesaian Problem 2 dibagi menjadi dua tahap perulangan setelah panjang pesan (*len*) dihitung secara manual dengan menelusuri karakter hingga menemukan tanda akhir string. Tahap pertama mengonversi setiap huruf pada pesan asli menjadi nilai numerik sesuai posisinya dalam alfabet (A=1 sampai Z=26) dan menyimpannya ke dalam array *origVal*. Nilai-nilai asli ini sengaja disimpan terlebih dahulu karena proses penyandian pada huruf ke-*i* membutuhkan nilai asli dari huruf sebelumnya (*i-1*), bukan nilai yang sudah disandikan, sesuai contoh pada soal.

kedua menghitung nilai huruf baru untuk setiap posisi. Huruf pertama (*i=0*) tidak mengalami perubahan sehingga nilai barunya sama dengan nilai aslinya. Untuk huruf-huruf berikutnya, nilai baru diperoleh dari penjumlahan nilai asli huruf tersebut dengan nilai asli huruf sebelumnya. Jika hasil penjumlahan melebihi 26, maka nilai tersebut dikurangi 26 agar tetap berada pada rentang 1 sampai 26, merepresentasikan aturan "jika hasil pergeseran melewati Z, perhitungan kembali dimulai dari A". Nilai akhir ini kemudian dikonversi kembali menjadi

karakter huruf dan disimpan ke array *result*, yang pada akhirnya ditampilkan sebagai pesan tersandi — sesuai dengan contoh pada soal di mana ALIENS menghasilkan AMUNSG.